

Node Internals

- **Node Internals**
 - **Inside Node.js Architecture (how node.js works)**
 - **What happens when you run node index.js**
 - **About Thread Pool**

Before proceeding further first guess the output of the below written two code

```
const fs = require('fs'); // 1

setTimeout(() => {
  console.log("SetTimeout done")
}, 0); // 2
setImmediate(() => {
  console.log("SetImmediate done")
}); // 3

console.log("Main work done"); // 4

// output ->
Main work done
SetTimeout done
SetImmediate done
```

But if i see the below code output

```
const fs = require('fs');

setTimeout(() => {
  console.log("SetTimeout done")
}, 0);
setImmediate(() => {
  console.log("SetImmediate done")
});

// output ->
SetImmediate done
SetTimeout done
```

can you see the difference just by using `console` the way code is running gets changed

Lets understand how this is working ??

Internally `node.js` to get async input or output (I/O) it uses `libuv`

read about `libuv` -> `libuv` and one more thing which `libuv` provides is **Thread Pool**

Inside `Node.js` Architecture (how `node.js` works)



`node.js` is a **single-threaded language**

now that does not means ki ye efficient nhi h understand by this example

say you have a restraunt and you have some situation like this

- **Case 1** -> you have 5 waiters who can take orders but there is a twist while taking the order they cannot do anything else once they will complete their order then only they will give it to chef
- **Case 2** -> In this you have only 1 waiter but he has the flexibility to take order and do some other task while taking it like giving it to chef



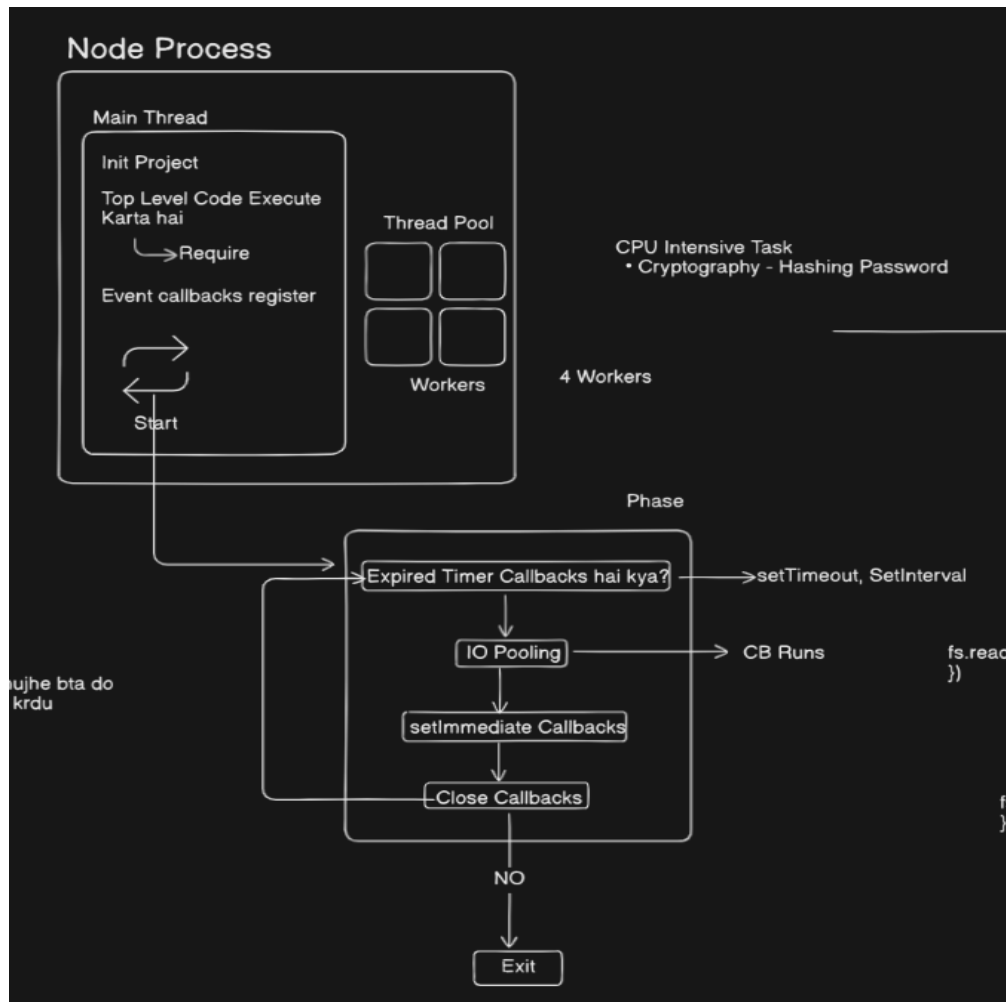
What do you think which is more good ??

Ans -> depends on the situation (if the cpu intensive task is going on then definitely **case 1** is good but when it comes to Rest APIs aur web APIs then **case 2** is good(`Node.js` belongs to this category))

.Bottleneck. -> a term widely used in software engineering referring to **Congestion** or simply saying **lot of traffic is being handled slowly** Term got its name from architecture of bottle (neeche se bda hota h but upar jake chota ho jata h to control the flow of liquid)

Ex -> `node.js` can act as **Bottleneck** for CPU intensive task.

What happens when you run `node index.js`



The only diagram you need to understand about the node.js architecture.

Thread Pool -> these are some of the waiters which are not doing anything

Main Thread -> This the main waiter that does most of the thing

Steps ->

1. Init Project -> Initialise the project (means now the project is ready to run)

2. Top Level code Execute -> Top level code wo code hota h jo **Global scope** me pda h kisi ka callback nhi h, module ke andar nhi h (main file ke andar global pe pda h) + Once **Top level code chلتa h** tb sbse phle **Require** statement chلتa h (take it as subpart of Top level code execute) as we have to use **Topological sort** (agar 'b' is dependent on 'a' to 'a' phle run krenge na) jitni v aur modules h wo chلenge

3. Event callbacks register -> jitne v events h unke andar pde **callbacks** register hote h taki jb ye events ho to ye callback chal jae

4. Event Loop -> what is event loop its something like while true loop only

```
while(true){ // yahan se aapka code execution start hoga

pehle mai ye krunga
then ye krunga
```

```
}
```

Inside Event loop (**Phases of Event Loop**)

1. Expired timer callbacks -> inside Event loop it first searches for **Expired timer callbacks** also known as **Task Queue** (koi aise callbacks h jo khatam ho gye h (means unka time khatam ho gya h) but their callback still waiting to be executed) agar h then these are **run** first - for **Expired timer callbacks** there are **only** two types of it

+ **setTimeout()** + **setInterval()**

2. I / O Pooling -> agar aapka code koi v **Input Output resource of mechanism** pe depend krta h to uske callback ko chlaya jata h ("**Fs**" **module**) now multiple Input output file operation are going on you know the syntax of **fs.readFile()** it takes a **callback function** when it reaches I / O polling it asks "**Kisi ka file wala circus complete ho gya ho to bta do**" jiska v ho jata h uska **callback** ko run kr do

3. setImmediate -> after this jitne v **setImmediate()** h unke callback ko run krega (if exists in the code)

4. Close Callbacks -> at last it run the **close callback** (means server ke run hone ke baad tm kuch krna chahte ho. Ex -> res.send type thing)

5. Finally it sees whether something is left if **yes** then chlo vapis upar if **No** to break kr jao (**EXIT**) - jb v **app.listen()** krte ho you will see terminal kbhi v band nhi hoga as the pending callbacks are still there (jitne v **Get**, **Post** register kiye h unke liye wait to krna pdega na **event Loop rok ke rkhta h**)

Now Talking about **Thread Pool**

all the above process were done on **main Thread** only then what is the use of **Thread Pool ??** -> helps to cater **CPU Intensive Task** (Ex -> cryptography (hashing, encoding, generating the password)) as these task if done in **Event Loop** will take time because of which there is a delay in the response so to handle these things, **Automatically** the main thread assign these task to available **Thread** from **Thread pool** if not available then **waits for it**

```
const fs = require('fs'); // 1

setTimeout(() => {
  console.log("SetTimeout done")
}, 0); // 2
setImmediate(() => {
  console.log("SetImmediate done")
}); // 3

console.log("Main work done"); // 4

// output ->
Main work done
SetTimeout done
SetImmediate done
```

Now decoding the sequence of code execution written at the starting

Step 1 -> sbse phle `require` chlega (`// 1`) as that is the top level (more priority code)

Step 2 -> then `// 4` chlega as that is the top level code now after this there are no more top level code in the file so

as there are no callback so skip the Events callback register step

Step 3 -> start **Event Loop** now first thing it searches for **koi v expired callback h or simply saying koi v type of expired timer callback h??** for us we have as `setTimeout()` to wo isko run krega

Step 4 -> at last if `setImmediate()` exists in code then uske `callback` ko run kr diya jayega



Based on the above concepts give the output to the below code ??

```
const fs = require('fs'); // step 1

setTimeout(() => console.log('setTimeout chla', 0)); // Step 4 -> as no more top
level code left so Event loop me chla gya ab aur sbse phle event callback register
hoke chla

setImmediate(() => console.log('setImmediate chla', 0)); // Step 6 -> IO Polling
ka pas response tha nhi to ye jo uska next step h(wo run kr gya) now after this
the next step is close callback but still IO Polling ka kaam bacha h to EXIT nhi
kr skte to wait for this (vapis upar se shuru kro phases of Event Loop)

// Step 2 -> 'fs' apna kaam krna start kr diya hoga
fs.readFile('sample.txt', 'utf-8', function(err, data){ // Step 5 -> Event
callback ke baad IO polling hona chahiye tha in our case 'fs' ne av tk complete
nhi kiya tha file read krna iske liye iska callback nhi chla jiska fayda iska next
step setImmediate callback utha liya aur wo chal gya ( Remember this is purely
subjective maybe kisi aur me chal gya ho av tk )
    setTimeout(() => console.log('file wala setTimeout chla'), 0);
    setImmediate(() => console.log('file wala setImmediate chla')); // Step 7 -
>dono setTimeout and setImmediate chenge immediately but as you are standing on
IO Polling ans iske baad setImmediate callback next step / phase h to setImmediate
phle run hua then jb wapas se start of phase of Event Loop pe gya to setTimeout
chla
})

console.log('hello'); // Step 3 -> as top level code to chlega he sbse phle

// Output

hello
setTimeout chla
setImmediate chla
file wala setImmediate chla
file wala setTimeout chla
```

⚠ Remember the above thing is **.Very Very Important.**

About Thread Pool

✈ **.Remember by default there are 4 threads in thread pool and they run parallelly(takes almost same time to complete any task).** But you can change it according to your needs by using **.process.env.UV_THREADPOOL_SIZE = your_desired_no.**(this functionality is by libUV)

see below to get better picture

The image shows two terminal windows side-by-side, illustrating the effect of the thread pool size on task execution time.

Left Terminal (Thread Pool Size = 4): The code defines a function to hash a password. It is called four times in parallel. The output shows that all four tasks complete almost simultaneously, with timestamps ranging from approximately 471ms to 480ms.

```

8 fs.readFile('sample.txt', 'utf-8', function (err, data) {
17
18   crypto.pbkdf2('password', 'salt1', 100000, 1024, 'sha512', (err, data) => {
19     console.log(`${Date.now() - start}ms: Password 2 Hashed`);
20   });
21
22   crypto.pbkdf2('password', 'salt1', 100000, 1024, 'sha512', (err, data) => {
23     console.log(`${Date.now() - start}ms: Password 3 Hashed`);
24   });
25
26   crypto.pbkdf2('password', 'salt1', 100000, 1024, 'sha512', (err, data) => {
27     console.log(`${Date.now() - start}ms: Password 4 Hashed`);
28   });
29 }

```

Right Terminal (Thread Pool Size = 1): The code is similar but includes two additional tasks (Password 5 and Password 6) after the first four. The output shows that the first four tasks complete almost simultaneously (around 491ms to 508ms), but then the execution pauses for a significant duration (around 955ms to 973ms) before the last two tasks (Password 5 and Password 6) complete, demonstrating a bottleneck when the thread pool size is 1.

```

8 fs.readFile('sample.txt', 'utf-8', function (err, data) {
30   crypto.pbkdf2('password', 'salt1', 100000, 1024, 'sha512', (err, data) => {
31     console.log(`${Date.now() - start}ms: Password 5 Hashed`);
32   });
33
34   crypto.pbkdf2('password', 'salt1', 100000, 1024, 'sha512', (err, data) => {
35     console.log(`${Date.now() - start}ms: Password 6 Hashed`);
36   });
37 }
38
39 console.log('hello');
40

```

in left part of image as by default 4 threads are there so they run parallelly and hence they are taking almost same time but in right part of image as we have added two more task so once all 4 completed their task (as they are running parallelly) the two wait and then after they(4) got completed the 2 gets runned in parallel.(so they have similar time)

Now after writing `process.env.UV_THREADPOOL_SIZE = 1` see the result one is waiting to get completed as it is the only one to do this task

The image shows a terminal window where the thread pool size is set to 1. The code defines a function to hash a password. It is called six times in sequence. The output shows that the tasks are executed sequentially, with timestamps increasing significantly for each task, demonstrating a bottleneck when the thread pool size is 1.

```

35 index.js > fs.readFile('sample.txt', 'utf-8') callback
4   setTimeout(() => console.log('Set Timeout'), 0);
5
6   setImmediate(() => console.log('Set Immediate'));
7
8   process.env.UV_THREADPOOL_SIZE = 1;
9
10  fs.readFile('sample.txt', 'utf-8', function (err, data) {

```

Output:

```

hello
Set Timeout
Set Immediate
Immediate inside FS 1
Set Timeout inside FS
[455ms]: Password 1 Hashed
[908ms]: Password 2 Hashed
[1357ms]: Password 3 Hashed
[1809ms]: Password 4 Hashed
[2260ms]: Password 5 Hashed
[2712ms]: Password 6 Hashed

```



The use of Thread Pool in node.js made it a very powerful run time environment